Análise de Vulnerabilidades em Programa C

Vinicius Trindade Dias Abel
Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG) Belo Horizonte – MG – Brasil

viniciustda@ufmg.br

1. Introdução

Este relatório detalha a análise do programa vuln.c, com o objetivo de identificar e corrigir vulnerabilidades de segurança. Foram utilizadas as ferramentas de compilação *gcc* com os sanitizers *AddressSanitizer (ASan)* e *UndefinedBehaviorSanitizer (UBSan)* para detectar erros de memória e comportamento indefinido em tempo de execução.

2. Análise dos Testes e Relatórios dos Sanitizers

Foram executados quatro testes distintos para avaliar o comportamento do programa.

Teste 1: Execução Normal

Este teste valida o funcionamento esperado do programa com entradas seguras.

Comando / Resultado:

```
trindade@trindade-G3-3500:~/Área de Trabalho/Cibersergurança$
 gcc -g -fsanitize=address,undefined -fno-omit-frame-pointer -o vuln vuln.c
trindade@trindade-G3-3500:~/Área de Trabalho/Cibersergurança$
 ./vuln "olá mundo" 11
Recebi: olá mundo
```

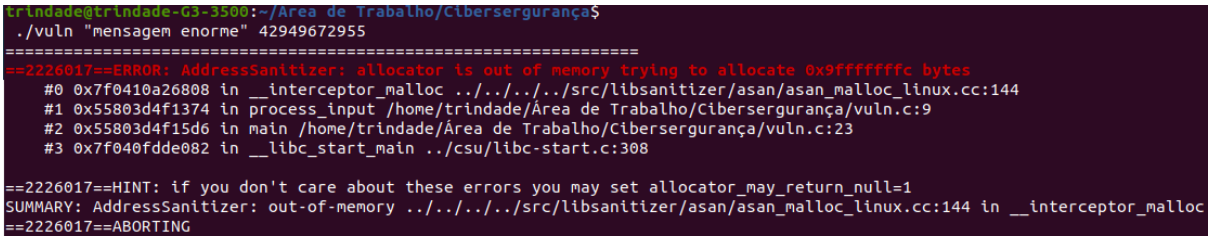
Análise:

A string "olá mundo" possui 10 caracteres. O count fornecido foi 11. O programa alocou count + 1 = 12 bytes. A função strcpy copiou 10 caracteres mais o terminador nulo (\0), totalizando 11 bytes, em um buffer de 12 bytes. Nenhuma violação de memória ocorreu.

Teste 2: Falha por Exaustão de Recursos

Este teste usou um número muito grande, mas que não causa um overflow, para testar a alocação de memória.

Comando / Resultado:



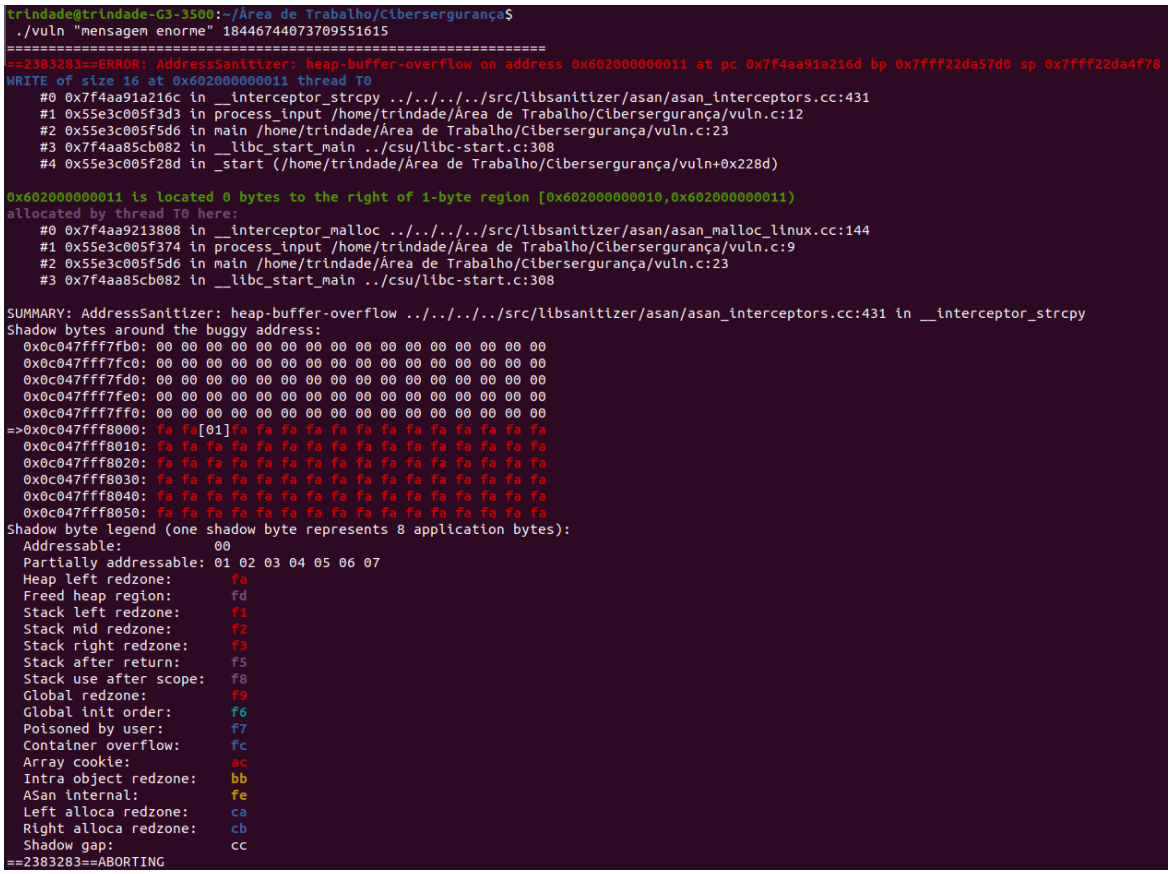
Análise do Erro:

- **O que aconteceu:** O programa tentou alocar 42.949.672.956 bytes (o count + 1), o que equivale a aproximadamente 42.9 GB de memória RAM.
- **Por que aconteceu:** O AddressSanitizer interceptou a chamada malloc e identificou que o sistema operacional não podia alocar essa quantidade de memória.
- **Significado:** Isso não é uma vulnerabilidade de corrupção de memória, como um overflow, mas pode ser uma vulnerabilidade de Negação de Serviço. Um usuário mal-intencionado poderia executar o programa com este comando para consumir toda a memória disponível e travar o sistema ou o processo.

Teste 3: Exploração de Integer Overflow

Este teste usou o valor máximo possível para um inteiro size_t de 64 bits, com o objetivo de causar um overflow.

Comando / Resultado:



Análise do Erro:

Este é o erro mais crítico. Ele ocorre em duas etapas:

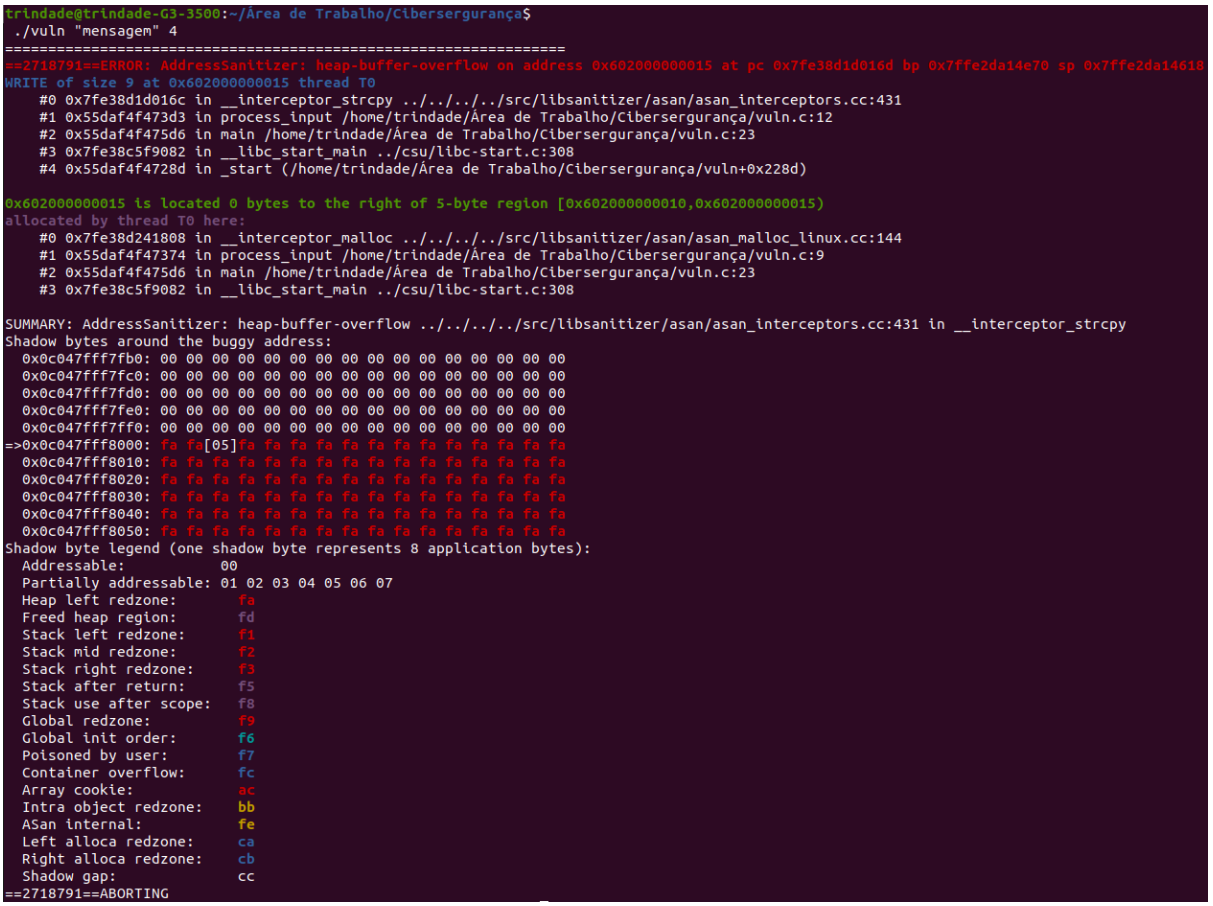
1. **Vulnerabilidade 1: Integer Overflow (Não reportado pelo UBSan, mas implícito)**
   - count recebe 18446744073709551615, que é o SIZE_MAX (valor máximo) para size_t em 64 bits.
   - Na linha malloc(total + 1), o programa calcula 18446744073709551615 + 1.
   - Essa soma resulta em um overflow e o resultado se torna **0**.
   - Portanto, o programa executa malloc(0).
2. **Vulnerabilidade 2: Heap Buffer Overflow (Reportado pelo ASan)**

- O ASan informa que alocou uma região de 1-byte (o malloc(0) pode retornar NULL ou um ponteiro para uma região de tamanho mínimo, como 1 byte).
- Em seguida, a função strcpy é chamada.
- strcpy tenta copiar "mensagem enorme" (que tem 16 caracteres, detectado pelo ASan como WRITE of size 16) para dentro do buffer de 1 byte.
- Isso causa uma escrita fora dos limites do buffer alocado (heap-buffer-overflow), corrompendo a memória.

Teste 4: Exploração de Buffer Overflow Clássico

Este teste prova que o strcpy é inseguro por si só, mesmo sem o integer overflow.

Comando / Resultado:



Análise do Erro:

- **O que aconteceu:** Count foi definido como 4. O programa alocou count + 1 = 5 bytes.
- **Por que aconteceu:** A string "mensagem" tem 8 caracteres (+ 1 \0 = 9 bytes). A função strcpy tentou copiar 9 bytes (detectado pelo ASan como WRITE of size 9) para dentro de um buffer de apenas 5 bytes.
- **Significado:** Isso confirma a **Vulnerabilidade 2**: O programa não usa a variável count para limitar a cópia de memória. Ele confia cegamente que a string de entrada caberá no buffer alocado, levando a um clássico heap-buffer-overflow.

3. Identificação dos Erros no Código-Fonte

Com base na análise dos sanitizers, identificamos dois pontos de falha principais no arquivo vuln.c:

1. **Vulnerabilidade 1: Integer Overflow**
    - **Código:** char *buf = malloc(total + 1);
    - **Explicação:** O cálculo do tamanho do buffer é vulnerável. Se um usuário fornecer o valor SIZE_MAX (o maior número possível para size_t, como visto no Teste 3), a operação aritmética total + 1 dá overflow e resulta em 0. O programa então tenta alocar malloc(0), mas o resto do código assume que um buffer gigante foi alocado com sucesso.
2. **Vulnerabilidade 2: Buffer Overflow**
    - **Código:** strcpy(buf, s);
    - **Explicação:** A função strcpy é intrinsecamente insegura porque não verifica o tamanho do buffer de destino (buf). Ela copia a string de origem (s) até encontrar um terminador nulo (\0), mesmo que isso signifique escrever além dos limites da memória alocada. Isso foi demonstrado no Teste 4 (buffer pequeno, string grande) e no Teste 3 (buffer de 0 bytes, qualquer string).

O código abaixo destaca onde essas falhas ocorrem:

```c
void process_input(const char *s, size_t count) {
    // erro: cálculo de tamanho pode overflowar
    size_t total = count * sizeof(char);

    // VULNERABILIDADE 1 (Integer Overflow):
    // Se 'total' for SIZE_MAX, 'total + 1' estoura e vira 0.
    char *buf = malloc(total + 1);

    if (!buf) { perror("malloc"); exit(1); }

    // erro: copia sem checar limites

    // VULNERABILIDADE 2 (Buffer Overflow):
    // 'strcpy' não sabe o tamanho de 'buf'. Ele copia 's' até
    // encontrar um '\0', mesmo que 's' seja maior que 'buf'.
    strcpy(buf, s);

    printf("Recebi: %s\n", buf);
    free(buf);
}
```

4. Código Corrigido (vuln_fixed.c)

Para solucionar ambas as vulnerabilidades, o código foi reescrito para incluir checagens de segurança antes das operações de alocação e cópia.

```c
// vuln_fixed.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h> // Necessário para SIZE_MAX

void process_input_safe(const char *s, size_t count) {

    // CORREÇÃO 1: Prevenção do Integer Overflow
    // Verifica se 'count' já está no valor máximo.
    // Se estiver, 'count + 1' causaria um overflow.
    if (count >= SIZE_MAX) {
        fprintf(stderr, "Erro: Valor de 'count' muito grande, causa overflow.\n");
        exit(1);
    }

    // O tamanho total necessário é 'count' bytes + 1 byte para o '\0'
    size_t total_size = count + 1;
    char *buf = malloc(total_size);
    if (!buf) {
        perror("malloc");
        exit(1);
    }

    // CORREÇÃO 2: Prevenção do Buffer Overflow
    // Substitui 'strcpy' por 'strncpy'.
    // Copia no máximo 'total_size - 1' (ou seja, 'count') caracteres.
    strncpy(buf, s, total_size - 1);

    // GARANTIA DE SEGURANÇA:
    // 'strncpy' não garante o terminador nulo se a string 's'
    // for maior que o buffer. Força o último byte a ser '\0'.
    buf[total_size - 1] = '\0';

    printf("Recebi (seguro): %s\n", buf);
    free(buf);
}

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "uso: %s <texto> <count>\n", argv[0]);
        return 1;
    }
    size_t count = (size_t) strtoul(argv[2], NULL, 10);

    // Chamando a função segura
    process_input_safe(argv[1], count);
    return 0;
}
```

5. Justificativa das Correções

1. **Prevenção do Integer Overflow (if (count >= SIZE_MAX)):**
    - Ao verificar se count já é o valor máximo (SIZE_MAX) antes de somar 1, impedimos a operação aritmética de dar overflow. Se a condição for verdadeira, o programa é interrompido com uma mensagem de erro clara, em vez de prosseguir com um valor corrompido (0).
2. **Prevenção do Buffer Overflow (strncpy e buf[total_size - 1] = '\0')**
    - A função *strcpy* foi substituída pela *strncpy*, que aceita um parâmetro de tamanho máximo.
    - total_size - 1 (ou count) é passado como o tamanho máximo de cópia. Isso garante que, mesmo que a string de entrada *s* seja enorme, apenas os primeiros count caracteres serão copiados.
    - A linha buf[total_size - 1] = '\0'; garante que, se a string *s* for maior que count, strncpy não adiciona o terminador nulo. Assim, o buffer *buf* sempre contenha uma string C válida, prevenindo erros em funções subsequentes (como o printf).